

Day04 Part A 学习文档 v1.0 : Runtime 如何思考 (How Runtime Thinks) - Prompt 不是 Context

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part A 正式学习文档。

Day03 已经完成 Runtime 架构设计、核心组件、Conversation 与 Message 数据模型。Day04 开始进入 Context Builder，但 Part A 的真正主题不是先写

ContextBuilder.build()，而是先建立一个更底层的世界观：Prompt 不是 Context，Agent 的核心不是 Prompt Engineering，而是 Runtime Engineering。

与上一节的联系

Day03-03 讨论的是 Runtime 内部最重要的数据模型：

Conversation
Message
Runtime Event

当时得到的核心结论是：

Conversation 保存的不是普通聊天记录，而是 Runtime 运行过程中发生过的事实事件。

Day04 Part A 继续追问：

如果已经有了 Conversation，为什么还需要 Context Builder？

这一节的核心结论是：

LLM 从来不是直接读取 Conversation，而是读取 Runtime 根据当前世界状态投影出来的 Context。

也就是说，Conversation 是 Runtime 的事实记录，Context 是 Runtime 在某一轮调用 LLM 时生成的投影，Prompt 只是这个投影在 Provider Request 中的一部分。

目录

Day04 为什么从 Context Builder 开始

Prompt 不是 Context

Context 也不是 Conversation

最终给 LLM 的不是一段 Prompt，而是一整个 Request

为什么复制最终 Prompt 磨不平 Agent 差异

Runtime 与 LLM 的职责边界

Tool Call 不是 Runtime 写满 if else

Tool、Workflow 与行业能力

Agent Runtime 与传统软件框架的类比

Runtime State 与 Projection

Context Builder 真正做什么

LLM 的智能与 Agent 的智能

Part A 核心结论

下一 Part 学习计划

写书 TODO

写书素材

本 Part 核心认知升级

1. Day04 为什么从 Context Builder 开始

Day04 最初的主题是 Context Builder，问题是：

为什么同一个 LLM，在 ChatGPT、Cursor、Claude Code、OpenAI Agents SDK 中表现得像完全不同的 AI？

直觉上，很多人会把差异归因于模型本身，或者归因于某个更好的 Prompt。

但 Part A 最重要的修正是：

答案不是模型本身，也不只是一段 Prompt，而是 Runtime 如何构建 Context。

同一个模型在不同产品里表现不同，是因为它每一轮看到的世界不同。

Cursor 能看到 Workspace、当前文件、选区、Git Diff、诊断信息、工具能力和项目约束；ChatGPT 看到的是聊天上下文、上传文件、记忆和可用工具；Claude Code 看到的是终端、文件系统、任务状态和代码执行结果。

模型可能相同，但 Runtime 投影出来的世界不同，最终表现就会不同。

2. Prompt 不是 Context

这一节从一个关键疑问开始：

如果我能拿到 Cursor 或 Claude Code 最终发送给 LLM 的 Prompt，再直接调用 OpenAI API，是否能磨平差异？

答案是：

能磨平一部分差异，但磨不平全部差异。

原因在于：

Prompt != Context

Prompt 更像是发给模型的一段文本或消息片段；Context 则是 Runtime 当前世界的一次组织结果。

Context 可能包含：

- System / Developer 指令；
- 用户当前输入；
- Conversation 历史；
- Summary；
- Memory；
- Workspace 状态；
- 当前文件、选区、错误信息；

- Git Diff ;
- Tool Schema ;
- Workflow 状态 ;
- Provider 约束 ;
- 安全策略与上下文压缩策略。

所以 Prompt 只是 Context 的一个可见表层，不是 Context 的全部。

更准确地说：

Prompt 只是 Runtime 当前状态的一张快照。

3. Context 也不是 Conversation

Day03 已经说明 Conversation 是 Runtime 的事实记录。

但 Conversation 不等于 Context。

Conversation 记录的是：

```
UserMessage
AssistantMessage
ToolCallMessage
ToolResultMessage
SummaryMessage
ErrorMessage
ApprovalMessage
Checkpoint
```

Context 是 Runtime 在某一轮调用 LLM 前，根据当前任务目标和上下文策略，从 Runtime State 中选择、裁剪、排序、转换后得到的投影。

可以这样理解：

```
Conversation = Runtime 发生过什么
Context      = 这一轮需要让 LLM 看见什么
Prompt      = Context 在 Provider Request 中的文本化表达之一
```

所以：

Conversation 保存事实，Context Builder 生成投影，LLM 看到的永远只是 Runtime 世界在某一时刻的一张快照。

4. 最终给 LLM 的不是一段 Prompt，而是一整个 Request

直接调用大模型 API 时，表面上像是：

```
User Prompt -> LLM -> Answer
```

但真实过程更接近：

```
Runtime State
|
v
Context Builder
|
v
Provider Request
|
v
```

LLM

Provider Request 里不只是用户输入，还可能包含：

- 多条 role message ；
- Tool definitions ；
- Tool choice ；
- Response format ；
- Model 参数 ；
- Safety / policy 信息 ；
- Stream 配置 ；
- Provider adapter 的格式转换。

因此，最终给 LLM 的不是一段孤立 Prompt，而是一整个请求对象。

这也是为什么“拿到最终 Prompt”只能复制一部分能力。你复制到的通常只是某一轮快照，而不是持续维护这个快照的 Runtime。

5. 为什么复制最终 Prompt 磨不平 Agent 差异

即使拿到了某一轮最终 Prompt，也仍然复制不了完整 Agent 能力，原因至少有五个。

第一，拿不到 Runtime State

Cursor 的能力不是来自某一段神秘 Prompt，而是来自它维护了一个与代码项目相关的世界状态：

```
Workspace
Current File
Selection
Diagnostics
Git Diff
Terminal Output
Tool Results
Task State
```

这些状态会随着用户操作和工具执行不断变化。

第二，拿不到动态更新过程

Agent 不是一次性调用模型，而是一个循环：

```
LLM decides
Runtime acts
State updates
Context rebuilds
LLM decides again
```

复制某一轮 Prompt，只复制了循环中的一个瞬间。

第三，拿不到工具执行能力

LLM 不会自己执行工具。它只能生成类似 Tool Call 的结构化意图。

真正执行工具的是 Runtime。

第四，拿不到 Workflow 和策略

专业 Agent 往往不只是有工具，还会有 Workflow、Context Policy、Memory Policy、权限控制、重试策略和错误恢复策略。

第五，拿不到 Provider Adapter

不同 Provider 对消息、工具、流式输出、结构化结果的协议不同。Context Builder 之后还需要 Provider Adapter，把内部 Context 转换为具体模型 API 能理解的请求。

所以：

真正难复制的是 Runtime，而不是某一轮 Prompt。

6. Runtime 与 LLM 的职责边界

Part A 中反复澄清了一个问题：

Runtime 做了这么多事，那 LLM 还智能在哪里？

关键在于 Runtime 和 LLM 做的是不同类型的工作。

LLM 负责：

- 理解
- 推理
- 规划
- 决策
- 生成

Runtime 负责：

- 维护状态
- 组织 Context
- 暴露 Tool
- 执行 Tool
- 控制 Loop
- 记录事件
- 处理副作用
- 适配 Provider

Tool 负责：

- 执行具体能力
- 返回结构化结果

所以，LLM 的智能不是体现在它亲自查天气、读文件、执行命令，而是体现在：

它知道什么时候应该调用什么工具，以及拿到工具结果以后下一步应该怎么办。

Runtime 的职责也不是替 LLM 思考，而是构建一个让 LLM 能正确思考的环境。

7. Tool Call 不是 Runtime 写满 if else

另一个重要疑问是：

如果 Runtime 要处理天气、地图、数据库、文件，那是不是要写无数 if else？

答案是：不是。

传统软件往往把流程写死：

```
if user_wants_weather:
    call_weather_api()
elif user_wants_stock:
    call_stock_api()
```

```
elif user_wants_email:
    send_email()
```

Agent Runtime 的设计不是提前枚举所有用户意图，而是向 LLM 暴露一组可组合的能力：

```
Tool Registry
Tool Schema
Tool Permission
Tool Result
```

LLM 根据当前目标和上下文选择工具，Runtime 只负责验证、执行、记录和返回结果。

因此：

Agent 的能力不是来自拥有所有 Tool，而是来自拥有少量通用、可组合的原子 Tool，再由 LLM 在 Runtime 中动态规划组合它们完成更多任务。

8. Tool、Workflow 与行业能力

Part A 中有一个很重要的认知升级：

行业能力不等于行业代码。

行业能力应该沉淀为：

- Tool；
- Workflow；
- Context Policy；
- Memory Policy；
- Domain Knowledge；
- Permission Boundary；
- Evaluation Rule。

例如医疗 Agent 不一定比 Claude Code 使用了更强的模型，但它可能拥有更适合医疗场景的 Runtime：

```
医学知识库
病历读取工具
诊疗流程约束
风险提示策略
用药安全规则
专业术语解释
审批与审计流程
```

这说明未来 Agent 产品的竞争不只是：

GPT vs Claude

而是：

```
同一个 LLM
|
不同 Runtime
|
不同产品体验
```

模型决定能力上限，Runtime 决定真实世界中的表现。

9. Agent Runtime 与传统软件框架的类比

为了理解 Runtime，可以把它类比为前端框架和操作系统。

与 React / Vue 的类比

传统互联网框架帮助开发者构建 Web 应用：

- State
- Component
- Lifecycle
- Render
- DOM

Agent Runtime 帮助开发者构建 AI 应用：

- Runtime State
- Context Builder
- Memory
- Tool
- Workflow
- Runtime Loop
- Provider Adapter

前端开发者不再手写所有 DOM 操作，而是组织状态和组件；Agent 开发者也不应该手写所有执行路径，而是组织工具、状态、上下文策略和工作流。

与操作系统的类比

如果把 LLM 类比为 CPU：

- LLM = CPU
- Runtime = Operating System
- Context Builder = Scheduler / Projection Layer
- Tool = Device / System Capability
- Prompt = 某一时刻送进 CPU 的指令

CPU 的能力决定计算上限，但操作系统决定它什么时候计算、计算什么、使用哪些资源。

同理，LLM 的能力决定推理上限，但 Runtime 决定它能看见什么、能调用什么、如何持续行动。

10. Runtime State 与 Projection

Part A 最后补齐的 20% 是 Runtime State 和 Projection。

要理解 Context Builder，必须先理解：

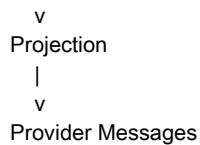
Runtime 维护着一个世界。

这个世界不是单一 Conversation，而是许多状态的组合：

- Conversation
- Memory
- Summary
- Workspace
- Current File
- Selection
- Git Diff
- Available Tools
- Workflow
- Instruction
- Metadata
- Planner State

Context Builder 的工作不是把所有状态一股脑塞给 LLM，而是做一次投影：

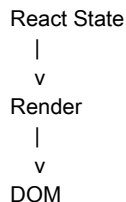
- Runtime State
- |



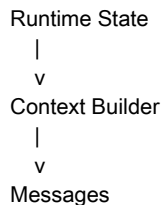
Projection 的意思是：

从一个更大的 Runtime 世界中，选出当前这一轮 LLM 最需要看见的部分，并转换为 LLM 能理解的形式。

这和 React 很像：



对应到 Agent Runtime：



11. Context Builder 真正做什么

Context Builder 不是简单拼 Prompt。

它更像一个 Context Compiler，负责把 Runtime 的内部状态编译成 Provider 可以接收、LLM 可以理解的输入。

它至少要处理这些问题：

- 哪些历史消息要保留；
- 哪些历史消息要压缩成 Summary；
- 哪些 Memory 要注入；
- 当前用户目标是什么；
- 当前工具能力有哪些；
- 当前 Workspace 状态哪些相关；
- 哪些内容不能进入 Context；
- 如何排序不同来源的信息；
- 如何适配不同 Provider 的消息格式；
- 如何在 Context Window 内保留最有价值的信息。

所以，Context Builder 的本质是：

在每一轮 Agent Loop 中，把 Runtime 当前世界重新组织成最适合 LLM 思考的一份 Context。

12. LLM 的智能与 Agent 的智能

学习过程中还讨论了一个更底层的问题：

LLM 只是预测 token，还能称为智能吗？

训练目标上的确是 Next Token Prediction，但这不能直接等同于“没有智能”。

大模型在足够规模和数据下会表现出：

- 推理
- 规划
- 抽象
- 归纳
- 迁移

这些可以理解为涌现能力。

但 LLM 本身并不生活在真实世界中。它不知道你的文件系统、项目状态、浏览器页面、数据库结果，也不能自己执行副作用。

因此，更准确的理解是：

LLM 是一种新的计算引擎，而 Agent Runtime 是围绕这种计算引擎诞生的新一代软件工程。

Agent 的智能来自组合：

- LLM 的推理能力
- Runtime 的世界建模能力
- Tool 的执行能力
- Workflow 的过程约束
- Memory 的长期积累

13. Part A 核心结论

Part A 最终形成了五条核心结论。

第一条

Prompt 不是 Runtime，而只是 Runtime 当前状态的一张快照。

第二条

Context 不是 Conversation，而是 Runtime 当前世界的一次投影。

第三条

LLM 从来没有看到 Runtime，它看到的永远只有 Runtime 投影出来的 Messages。

第四条

Runtime 的职责不是替 LLM 思考，而是在每一轮循环中，把当前世界重新组织成最适合 LLM 思考的 Context。

第五条

Agent 的智能不仅来自 LLM，也来自 Runtime 如何维护世界状态，并持续把世界状态投影给 LLM。

可以把 Part A 总结成一句话：

Agent Runtime 的职责不是替 LLM 做决策，而是构建一个让 LLM 能够正确思考和行动的世界。

14. 下一 Part 学习计划

下一 Part 是：

Day04 Part B : Runtime State - Agent 到底维护着一个怎样的世界？

Part B 的学习目标是从 Prompt 世界正式进入 Runtime 世界。

主要问题包括：

Runtime State 是什么？

Runtime 为什么需要维护 State？

State 和 Conversation 有什么区别？

Runtime 到底维护哪些 State？

谁创建 State？

谁修改 State？

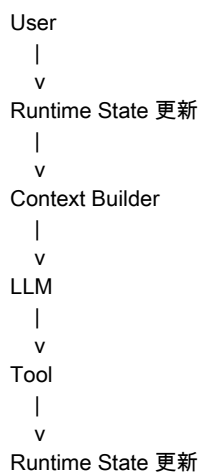
State 的生命周期是什么？

State 是否持久化？

State 是否进入 Context？

Runtime State 与 React State 如何类比？

Part B 的核心链路会是：



15. 写书 TODO

TODO 1：新增章节“Prompt 不是 Context”

这一章应该作为 Runtime 部分的开场。

它负责打破最常见的误解：

Prompt = AI 应用的全部

更准确的表达是：

Prompt = Runtime State 的一次快照

TODO 2：引入 Projection 思想

不要再把 Context Builder 描述成“拼 Prompt”。

更准确的说法是：

Context Builder 将 Runtime State 投影成 LLM 可以理解的世界。

Projection 应该成为整本书的核心概念之一。

TODO 3：增加 Runtime 与 LLM 的职责划分图

建议保留如下结构：

LLM
负责：
理解
推理
规划
决策

Runtime
负责：
维护状态
组织 Context
调度 Tool
控制 Loop

Tool
负责：
执行能力

TODO 4：增加 Runtime 类比章节

建议增加两个类比：

- Agent Runtime 与 React Runtime；
- Agent Runtime 与 Android / Operating System。

这些类比可以帮助前端工程师快速建立直觉。

TODO 5：保留一句全书主线

Agent Runtime 的职责不是替 LLM 思考，而是构建一个让 LLM 能够正确思考的世界。

16. 写书素材

以下句子可以作为后续写书素材。

素材一

Prompt 只是 Runtime 当前状态的一张快照。

素材二

Context 并不是 Conversation，而是 Runtime 世界的一次投影。

素材三

Runtime 不负责替 LLM 做决策，而负责创造一个适合 LLM 做决策的环境。

素材四

模型决定推理能力的上限，Runtime 决定 Agent 在真实世界中的表现。

素材五

传统软件是在代码中固定用户的执行路径；Agent 软件是在 Runtime 中提供能力边界，让 LLM 在边界内动态规划执行路径。

素材六

行业能力并不等于行业代码，而是将行业知识沉淀为 Tool、Workflow、Context Policy 和 Memory Policy，让 LLM 能够像该领域的专家一样思考和行动。

素材七

LLM 并不是一个生活在真实世界中的智能体，它更像一个拥有强大推理能力的大脑；而 Agent Runtime 的职责，是为这个大脑构建感知世界、理解世界和行动世界的能力。

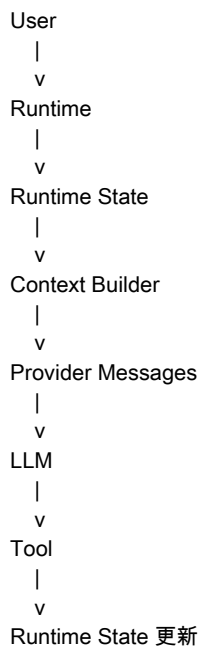
17. 本 Part 核心认知升级

学习前

Prompt -> LLM -> Answer

这种理解把 AI 应用看成一次模型调用。

学习后



这种理解把 AI 应用看成是一个持续运行的系统。

最大的认知升级是：

Agent 的核心不是 Prompt Engineering，而是 Runtime Engineering。